

Answer key and discussion

Instructor copy - keep separate from the student worksheet.

Page 1: Be the function

Round A: number stays 4. The local `n` starts at 4, becomes 8, and the function returns 9. result becomes 9. After the call ends, `n` is no longer available to the caller.

Round B: the call follows the same steps and returns 9. The assignment `number = change(number);` stores 9 in `number`. Assigning to `n` changes only the local copy of the primitive `int` value.

Page 2: Predict, run, explain

Output: 5, 6, 7, 6. The first returned 6 is discarded. The assignment stores 6 in `a`. Printing `addOne(a)` prints 7 without changing `a`, so the last line prints 6.

One valid change: replace the first `addOne(a);` with `a = addOne(a);`. The outputs then become 6, 7, 8, 7. Accept other single-statement changes that leave `a` equal to 7 and a correct explanation.

Page 3: Function repair shop

A compiles in Processing Java mode, but reads `mouseX` instead of `value`. A repair is `return value * 2;`. Test `twice(3) -> 6` and `twice(-2) -> -4` while moving the mouse: the results should stay fixed.

B does not compile: an `int` function has no returned value. Replace `println(n * n);` with `return n * n;`. Test `square(4) -> 16`, `square(-3) -> 9`, `square(0) -> 0`. Print or store the result in the caller.

C does not compile: when `n <= 0`, execution reaches the end without returning a boolean. A concise repair is `return n > 0;`. Tests: `-2 -> false`, `0 -> false`, `3 -> true`. An `if/else` returning both values is also valid.

Page 4: Planning and exit question

Look for functions with clear responsibilities, meaningful parameters, and a returned value actually used by the caller. Tests should include a stated expected outcome, especially at a boundary.

Drawing changes the canvas as an effect. Returning supplies a value to the caller; it does not automatically draw, print, or update the caller's variable. A function may have both effects and a return value, so these are distinct behaviors rather than mutually exclusive categories.

Suggested 3.5-hour workshop

Timing is flexible. Use explanations and boundary tests to assess understanding.

0-15 min | Recall and predict

Review familiar calls. Distinguish a definition from a call.

15-40 min | Explain and trace

Use lesson sections 1-6, with worksheet page 2 as the return-value checkpoint.

40-60 min | Be the function

Worksheet page 1 in pairs. Ask which assignment changes the caller's variable.

60-70 min | Break

70-95 min | Repair shop

Worksheet page 3. Compare compile errors with incorrect results.

95-125 min | Scared Square

Focus on the separation between checking, drawing, and moving.

125-135 min | Break

135-155 min | Refactor and explain

Preserve three observable behaviors while removing duplication.

155-200 min | Open studio

Plan on worksheet page 4, build, and pause for a partner call trace.

200-210 min | Share and exit

Show one design decision and one resolved bug; answer the exit question.

For students needing more support, use Exercise D as the studio starting point. Offer the boolean-function challenge to early finishers. These are choices, not a requirement to complete every activity.